

Это отличный материал! Видно, что проделана большая работа по структурированию логики. Такая инструкция — это буквально «золотой стандарт» для разработчика или технического интегратора.

Ниже я переоформил твой текст в чистый, структурированный формат, который идеально подходит для копирования в **Google Docs**, **Notion** или сохранения в **PDF**.

Полное руководство: n8n + OpenClaw Bot

Интеграция через Webhook и Skill

Это самодостаточная инструкция по созданию моста между AI-ассистентом и бэкэнд-автоматизацией.

1. Общая архитектура

Прежде чем настраивать, важно понять роли каждого компонента:

Компонент	Роль	Описание
OpenClaw Bot	Интеллект	Принимает команды, анализирует контекст и решает вызвать Skill.
Skill (Навык)	Кнопка действия	Формирует и отправляет JSON-запрос во внешний сервис.
n8n	Мозг / Исполнитель	Принимает запрос, проверяет безопасность, выполняет логику.

Ключевая идея: OpenClaw не перегружен логикой. Он делегирует тяжелую работу в n8n, работая как интерфейс.

2. Настройка n8n (Backend)

Шаг 1. Безопасность

Придумай секретный ключ (например, длинную строку из букв и цифр). Он будет передаваться в заголовке `x-n8n-secret` для защиты твоего Webhook от посторонних.

Шаг 2. Webhook Node

1. Создай узел **Webhook**.
2. **HTTP Method:** POST.
3. **Path:** `openclaw/skill1`.
4. **Важно:** Для работы используй **Production URL**.

Шаг 3. Проверка секрета (If Node)

Добавь узел **If** сразу после Webhook, чтобы отсеять неавторизованные запросы.

- **Expression:** `{{ $json.headers?.["x-n8n-secret"] || $headers["x-n8n-secret"] || $headers["X-N8N-SECRET"] }}`
- **Condition:** Equal to → ТВОЙ_СЕКРЕТНЫЙ_КЛЮЧ.

Шаг 4. Обработка веток (True/False)

- **False (Ошибка):** Добавь узел **Respond to Webhook**.
 - Response Code: 401.
 - JSON: `{"ok": false, "error": "unauthorized"}`.
- **True (Успех):** Добавь узел **Set** (или Edit Fields).

Шаг 5. Формирование ответа (Set Node)

В ветке **True** используй следующий JSON-код в режиме Expression:

JavaScript

```
{{
  (
    ok: true,
    message: "skill executed in n8n",
    received: {
      skill: $json.body?.skill,
      sessionKey: $json.body?.sessionKey,
```

```
    args: $json.body?.args
  },
  result: {
    pong: true,
    ts: Date.now(),
    summary: `Привет, ${$json.body?.args?.username}! Цель: ${$json.body?.args?.goal}, ниша:
${$json.body?.args?.niche}`
  }
})
}}
```



3. Настройка OpenClaw (Skill)

Чтобы зарегистрировать навык, отправь OpenClaw следующий промпт:

Prompt для создания Skill:

"Create a new skill named 'ping'.

Webhook URL: <https://твой-адрес.railway.app/webhook/openclaw/skill1>

Headers: Content-Type: application/json, x-n8n-secret: ТВОЙ_СЕКРЕТ

Request body format:

```
{
  "skill": "ping",
  "sessionKey": "<sessionKey>",
  "args": <args>
}
```

Rules: args must be a JSON object, do NOT stringify, send valid JSON only.

Create, register and activate this skill."

4. Тестирование и Демо

Как проверить вручную (Postman/cURL)

Отправь POST-запрос на свой URL с заголовком `x-n8n-secret` и телом:

JSON

```
{
  "skill": "ping",
  "sessionKey": "test:manual",
  "args": {
    "username": "igorzuevich",
    "goal": "продажи",
    "niche": "маркетинг"
  }
}
```

Сценарий для демо (Wow-эффект)

1. В чате с OpenClaw скажи: *"Запусти навык ping для пользователя igorzuevich, цель продажи, ниша маркетинг"*.
2. Переключись на экран **n8n**.
3. Покажи, как в режиме реального времени прилетает Webhook и формируется персонализированный ответ.

Что развивать дальше?

- **Switch Node:** Один Webhook может обрабатывать десятки разных скиллов, если поставить узел Switch по полю skill.
- **Database:** Сохраняй args в Notion или Google Sheets прямо во время выполнения скилла.
- **Notification:** Добавь узел Telegram в n8n, чтобы получать уведомление каждый раз, когда AI использует навык.

Хочешь, я помогу тебе составить более сложную логику для узла Switch, чтобы один вебхук мог управлять целым набором разных навыков?